



**TECNOLÓGICO
NACIONAL DE MÉXICO**

TECNM

**Tecnológico Nacional de México
Campus Culiacán**

**Ingeniería en Sistemas computacionales
Tópicos de Inteligencia artificial
12:00 – 13:00**

**Sistema Integral de Detección de Matrículas y Gestión de Incidencias para
el Instituto Tecnológico de Culiacán**

Alumnos:

**Caro García Jorge Ariel
Uribe Armenta Javier**

Docente:

**ZURIEL DATHAN MORA FELIX
28/11/2025**

Índice

Objetivo general.....	3
Objetivos Específicos.....	3
Descripción del Problema.....	3
Justificación.....	3
Manual de Usuario:.....	5
Instrucciones de Uso	5
Sección Usuarios	6
SecciónCámara	10
Sección Incidencias	14
Manual de Instalación.....	28
Documentación Técnica.....	29
Backend:	29
Frontend:	30
Lo importante por mencionar es el procesamiento del modelo de detección de placas	31
Proceso de detección de caracteres con PaddleOCR (backend)	32
Integración de ambos procesos en la aplicación	33
Información acerca del modelo	34
Modelo Relacional.....	35
Conclusiones.	36

Objetivo general

Se busca desarrollar un sistema móvil y backend, con el cual operadores en la institución puedan gestionar accesos e incidencias escaneando matriculas vehiculares, reconocer texto mediante IA y relacionarlas con los datos guardados en una base de datos institucional, esto con el fin de optimizar el control vehicular, la seguridad dentro de la institución así como llegar a la solución de incidencias en el Tecnológico de Culiacán.

Objetivos Específicos

- Debido a la inseguridad al acceder a la institución se busca optimizar el control de quien entra y sale de la misma con mayor precisión.
- Implementación de una base de datos relacional con la cual se gestionarán usuarios, vehículos e incidencias
- Monitoreo de las actividades vehiculares dentro de la institución.
- Diseñar e implementar un modelo de visión artificial (PaddleOCR) para detección y lectura de placas vehiculares.
- Desarrollar una aplicación móvil en Flutter en la cual se tomará foto de las placas vehiculares para el escaneo.
- Desarrollar un backend con FastAPI para procesar OCR, gestionar usuarios, vehículos e incidencias.
- Automatizar la relación entre las placas detectadas y propietarios registrados.
- Implementar un sistema de notificaciones por correo electrónico para incidencias aprobadas o rechazadas.
- Dentro de las incidencias permitirle a un administrador el rol de gestionarlasy ya sea el rechazar o aprobar, al igual de gestionar el estado del vehículo en cuanto a su acceso.
- Diseñar interfaces intuitivas, organizadas y fáciles de usar para administradores y operadores.

Descripción del Problema

El Instituto Tecnológico de Culiacán en este momento pasa por dificultades en el control de acceso vehicular y en la gestión de incidencias dentro del campus como por ejemplo el que se estacionen en lugares prohibidos o la entrada de vehículos no autorizados. Esto se debe a que actualmente no existe un sistema automatizado para identificar los vehículos que acceden, ya que la verificación de placas como tal se realiza de forma manual, además no hay un registro unificado de incidencias de tráfico interno ya que los guardias deben tomar los datos manualmente, lo que provoca errores y retrasos en la solución de incidencias. Lo que nos lleva concluir que la institución necesita una herramienta moderna para optimizar el flujo vehicular y mejorar la seguridad, proponemos un sistema integral que automatiza estos procesos mediante visión artificial y administración digital.

Justificación

El presente proyecto es de relevante importancia porque mejoraría la seguridad institucional, permitiendo identificar vehículos registrados y detectar vehículos no autorizados, además automatiza procesos que en la actualidad los llevan manualmente, reduciendo errores y tiempos de espera ya sea en el acceso o al llevar una incidencia mediante evidencias con

fotos, ubicación y datos automáticos. Brinda información en tiempo real lo que reduce la carga operativa del personal y agiliza los procesos. Da el paso a modernizar el proceso de registro vehicular alineándose con tecnologías actuales (IA, OCR, apps móviles, APIs, Docker), apoyando así a la toma de decisiones administrativas más precisas mediante registros, conteos de incidencias y estatus automatizados. En su esencia se integra plenamente al contexto del Instituto Tecnológico de Culiacán, porque precisamente fue creado para este respondiendo a una necesidad real de seguridad y organización vehicular.

Manual de Usuario:

Instrucciones de Uso

Existen dos tipos de perfiles:

- Admin: Tiene todos los permisos: Añadir/Eliminar Usuarios y Autos, Ver TODAS las incidencias, Generar/Aprobar/Rechazar Incidencias
- Usuario Normal: Tiene acceso limitado: Solo puede generar incidencias y ver sus propias incidencias

El acceso a la función de detección de placa mediante la cámara y el de incidencias es global.

Al abrir la aplicación se mostraba la pantalla de inicio de sesión.



Figura 1: Pantalla de Inicio de Sesión

Para acceder se debe de ingresar el número de control del usuario, en caso de no tener, un administrador debe de crearle su perfil

Sección Usuarios



Figura 2: Pantalla de Usuarios

Como se puede observar, el administrador puede acceder a los usuarios registrados así, como agregar y borrarlos. Además, se tiene un buscador, para filtrar a los usuarios por el dato que se desee (Número de Incidencias, Estatus, No. Control, Nombre, etc.)

10:54 AM

← Agregar usuario

Nombre
Ariel Caro

Número de control
21170276

Edad
21

Correo
javieruribe00@outlook.com

Guardar usuario

Figura 3: Pantalla de Agregar Usuario

Al darle click al botón de agregar usuario, se abrirá esta pantalla, para llenar los datos del usuario, el nombre, el número de control, la edad y el correo. (El estatus del usuario y el No. de Incidencias se definen con un valor predeterminado de 'Autorizado' y 0 respectivamente, este va a cambiar automáticamente conforme el flujo de la aplicación.

Al darle “Guardar Usuario” el usuario se añadirá a la Base de Datos y estará disponible en el sistema (Figura 1)

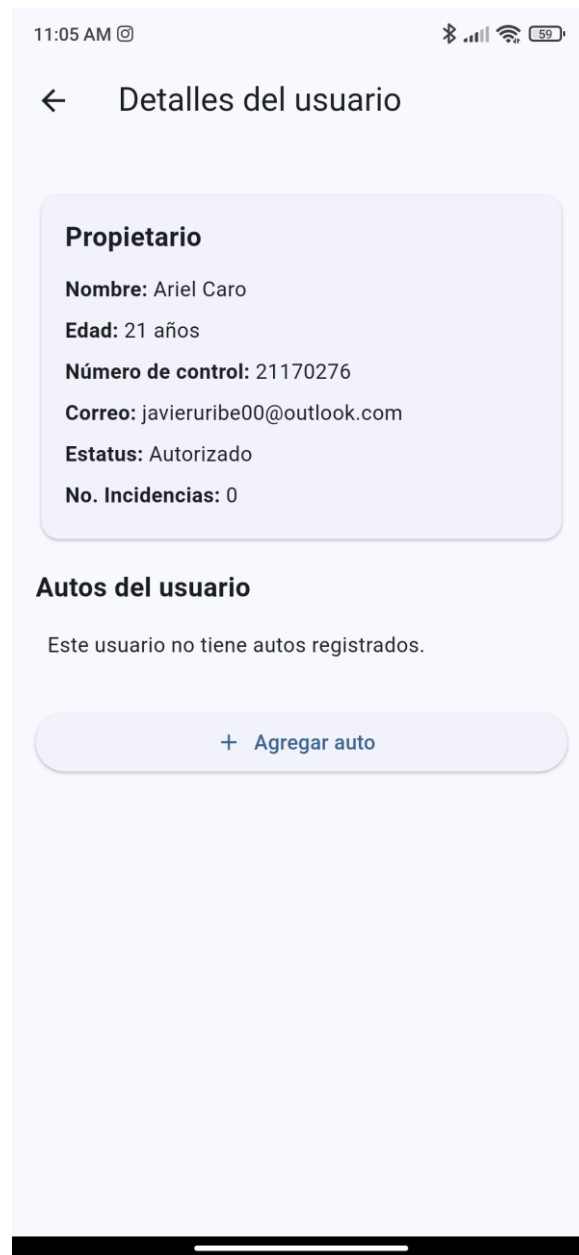


Figura 4: Pantalla de Detalle de Usuario

Al pulsar en algún usuario se mostrarán los datos del usuario, a su vez un botón para agregar un auto.

11:07 AM    

← Agregar auto

Placa

Marca

Modelo

Color

 Guardar auto

Figura 5: Pantalla de Agregar Auto

Al pulsar el botón de Agregar Auto, se abrirá la pantalla para agregar los datos del auto, la placa (esta debe ser única, no debe repetirse), la marca, el modelo y el color.

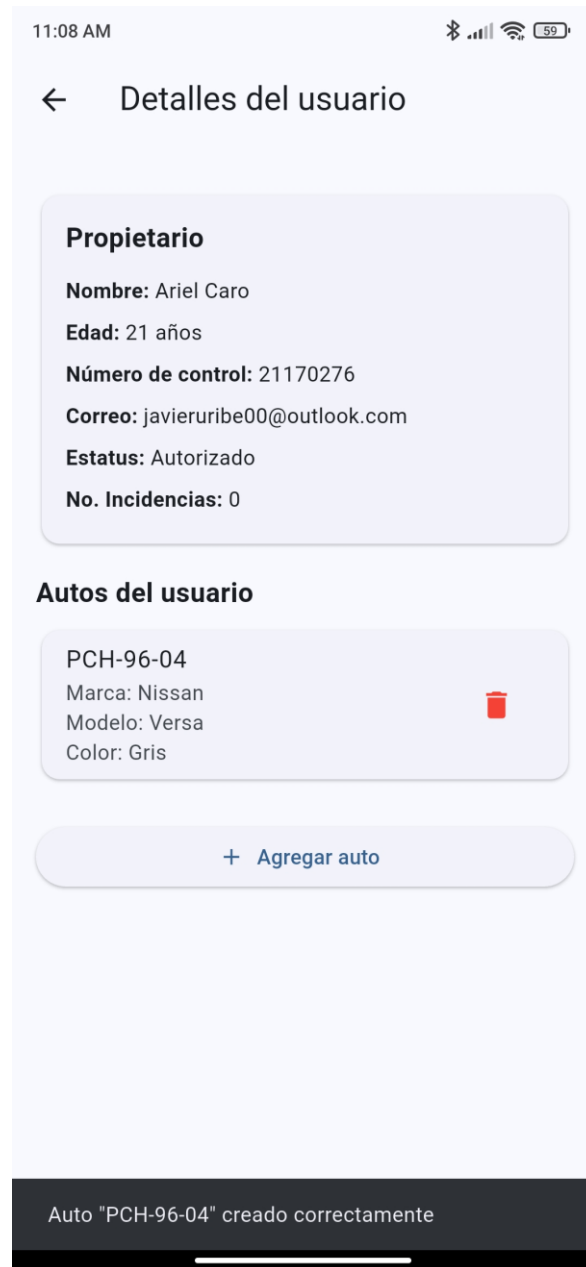


Figura 6: Pantalla de Detalle de Usuario con auto agregado

Al guardar el auto, aparecerá en la sección de detalles del usuario, y ahora ya será posible escanear la placa y obtener los datos de esta con la cámara.

Sección Cámara

Esta sección está disponible para ambos perfiles. Se debe ir a la opción Cámara de la aplicación

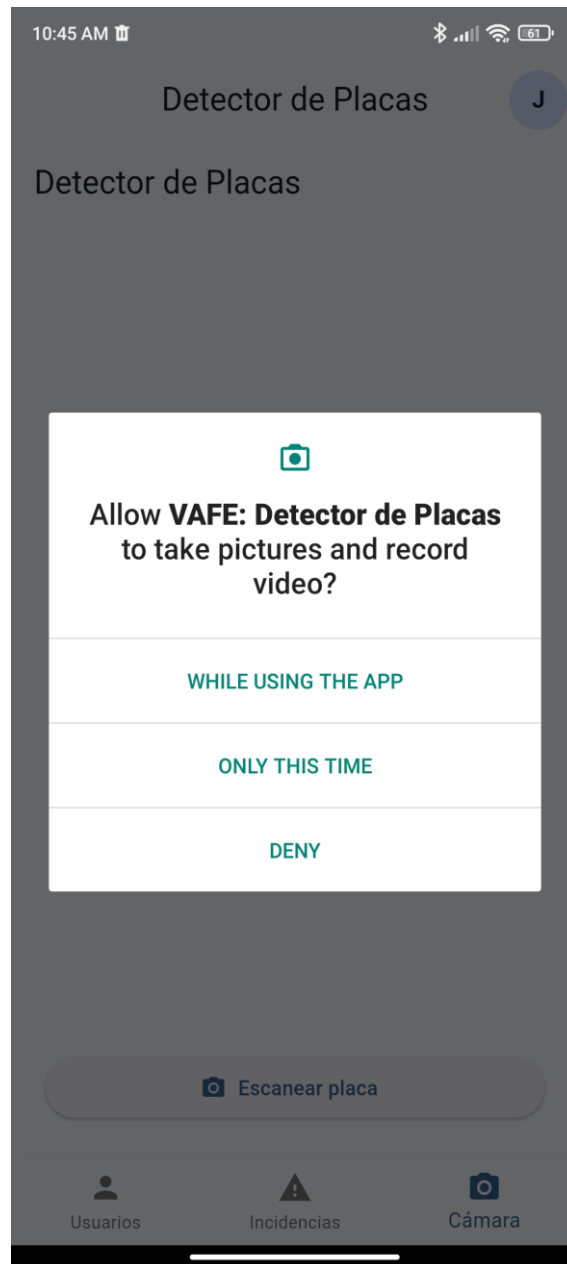


Figura 7: Pantalla de la Cámara Solicitud de Permisos

Una vez en esa sección, se deben de aceptar los permisos para usar la cámara, caso contrario, la función de la cámara no funcionará.



Figura 8: Pantalla de Camara (Detector de Placas)

Una vez dados los permisos, se mostrará la cámara con un rectángulo que es donde mas o menos se debe pocisionar la placa, y un botón para tomar la foto y escanear la placa.



*Figura 9: Pantalla de Detalles de Usuario
(Detección de Placa)*

Una vez detectada y de estar la placa registrada en la base de datos, se mostrará la información del usuario y autos pertenecientes a la placa escaneada.

Sección Incidencias

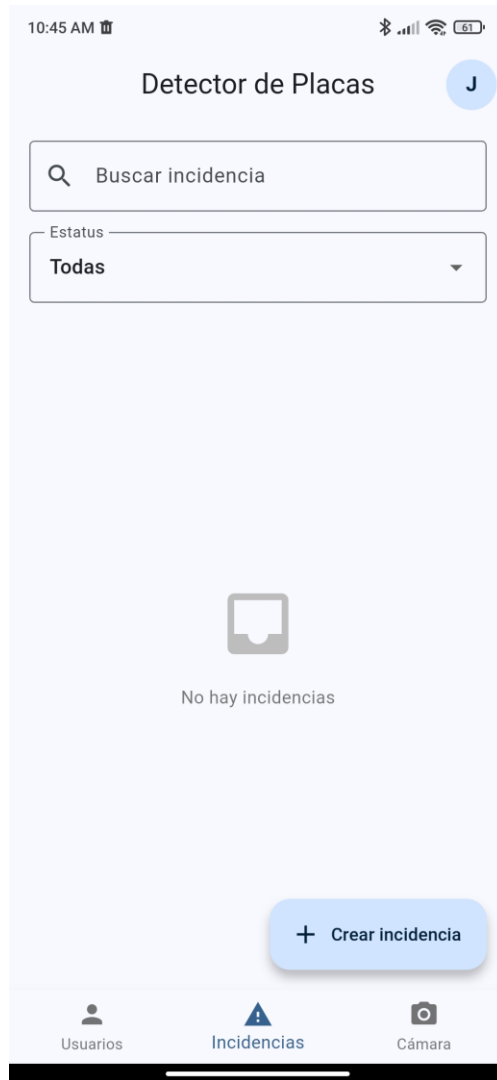
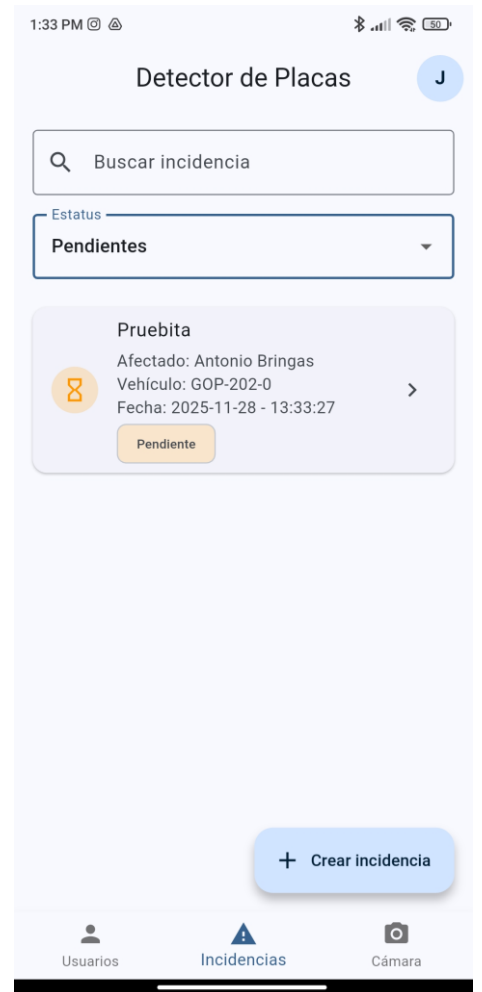
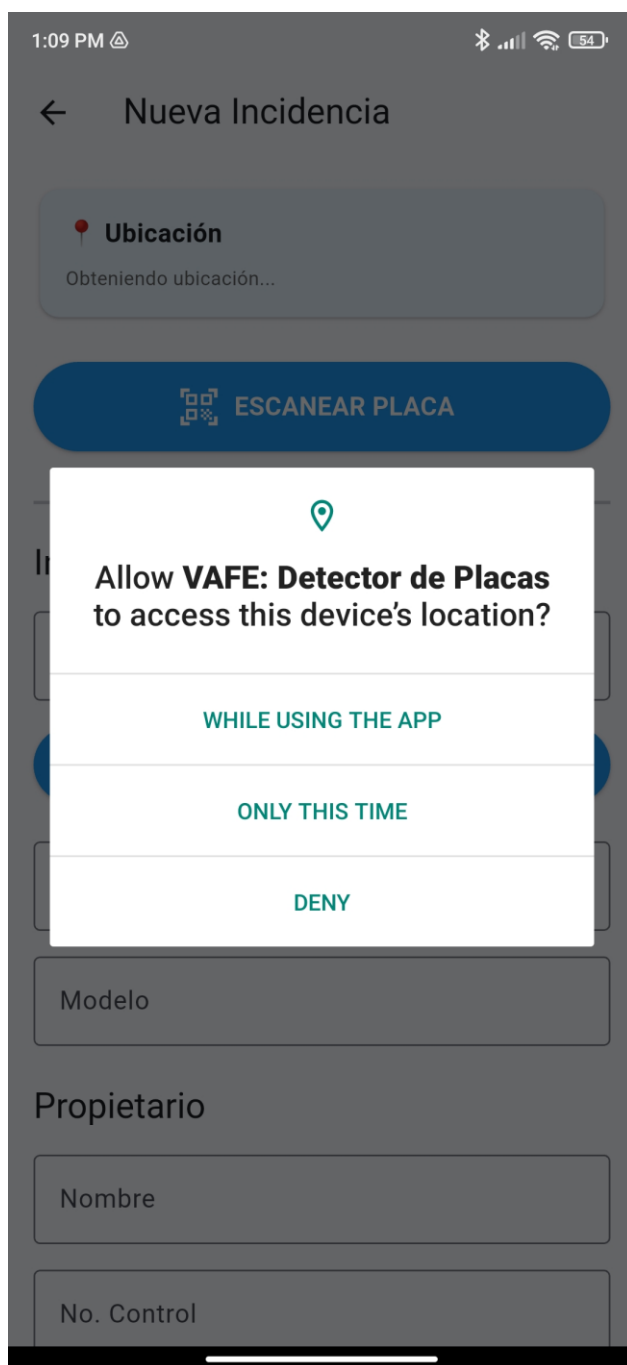


Figura 10: Pantalla de Sección de Incidencias

En esta pantalla se muestran las incidencias creadas por el usuario y un botón para crear una nueva incidencia (De ser administrador se muestran todas las incidencias creadas, así como un combobox con filtros para buscar entre todas las incidencias, las aprobadas, rechazadas y pendientes).



Al apretar el botón de crear incidencia se abrirá esta pantalla:



Se le debe de dar permisos para obtener la ubicación, para de esta forma poner la latitud y longitud en el registro de incidencias.

1:09 PM

 54

←

Nueva Incidencia

 **Ubicación**

Lat: 24.767883, Lon: -107.4610259

 **ESCANEAR PLACA**

Información del Vehículo

Placa

 **Buscar por Placa**

Marca

Modelo

Propietario

Nombre

No. Control

Al dar permisos, arriba se muestra la latitud y longitud en la que el usuario se encuentra actualmente.

Para llenar los datos se puede hacer de dos maneras, escanear la placa con la cámara, o también buscarla manualmente, ingresando el texto en el campo y presionando el botón Buscar por Placa. Sin importar el método, mientras la placa esté registrada, los campos se autocompletarán (aunque no aparezcan todos los datos ahí, los datos de la incidencia se llenan con base a la placa, los datos que se autocompletan son solo para mostrarle al usuario los datos de la placa).

1:12 PM

Nueva Incidencia

ESCANEAR PLACA

Información del Vehículo

Placa
DSZ-999-Z

Buscar por Placa

Marca
Lamborghini

Modelo
Aventador

Propietario

Nombre
Javier Uribe Armenta

No. Control
21170497

Evidencia (1/5) Agregar

1:15 PM

Nueva Incidencia

Modelo
Aventador

Propietario

Nombre
Javier Uribe Armenta

No. Control
21170497

Evidencia (2/5) Agregar

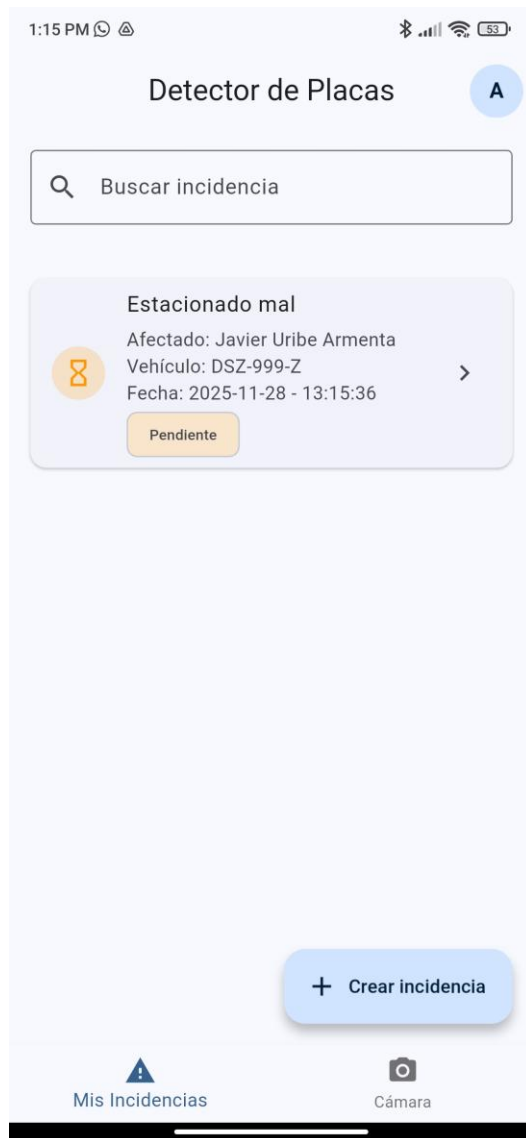
DSZ-999-Z

Estacionado mal

Descripción de la incidencia

Crear Incidencia

Más abajo se pueden agregar imágenes presionando el botón “Agregar” para tomar fotos con la cámara, y la descripción de la incidencia, una vez llenados los datos se debe presionar el botón “Crear incidencia”



Una vez creada, se mostrará la incidencia creada en la sección de incidencias, el siguiente paso es que un administrador rechaze o apruebe la incidencia.

1:15 PM 53

← Detalle de incidencia

Estatus Pendiente

Fecha y Hora: 2025-11-28 - 13:15:36

Descripción: Estacionado mal

Persona Afectada

Nombre: Javier Uribe Armenta

No. Control: 21170497

Correo: javier.ua01@gmail.com

Estatus: Autorizado

Incidencias totales: 0

Reportante

Nombre: Ariel Caro

No. Control: 21170276

Correo: javieruribe00@outlook.com

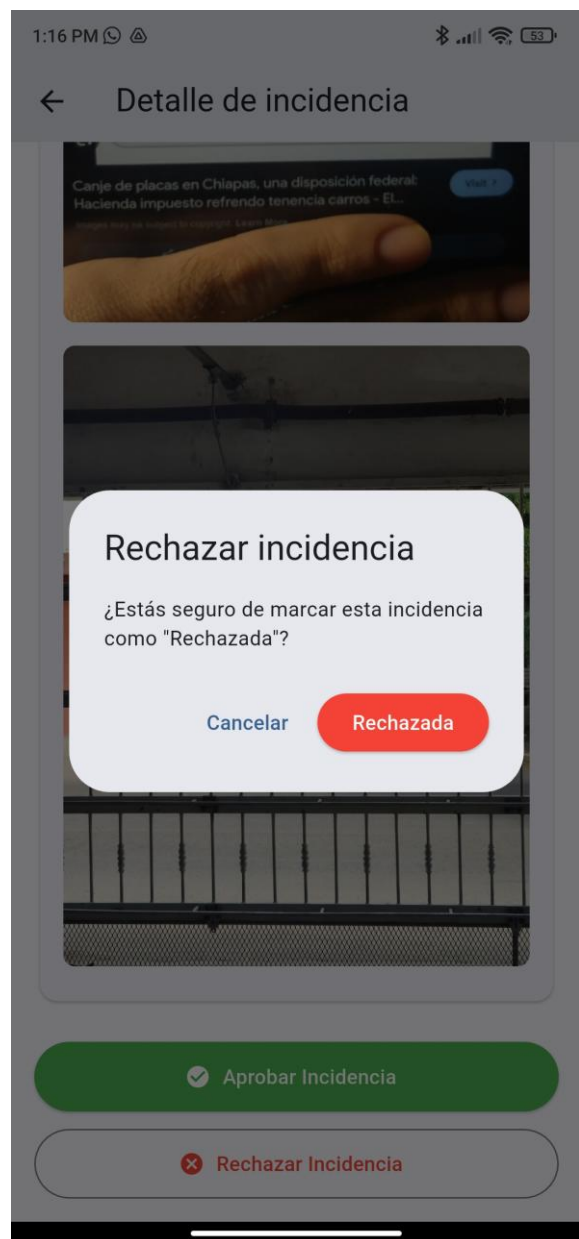
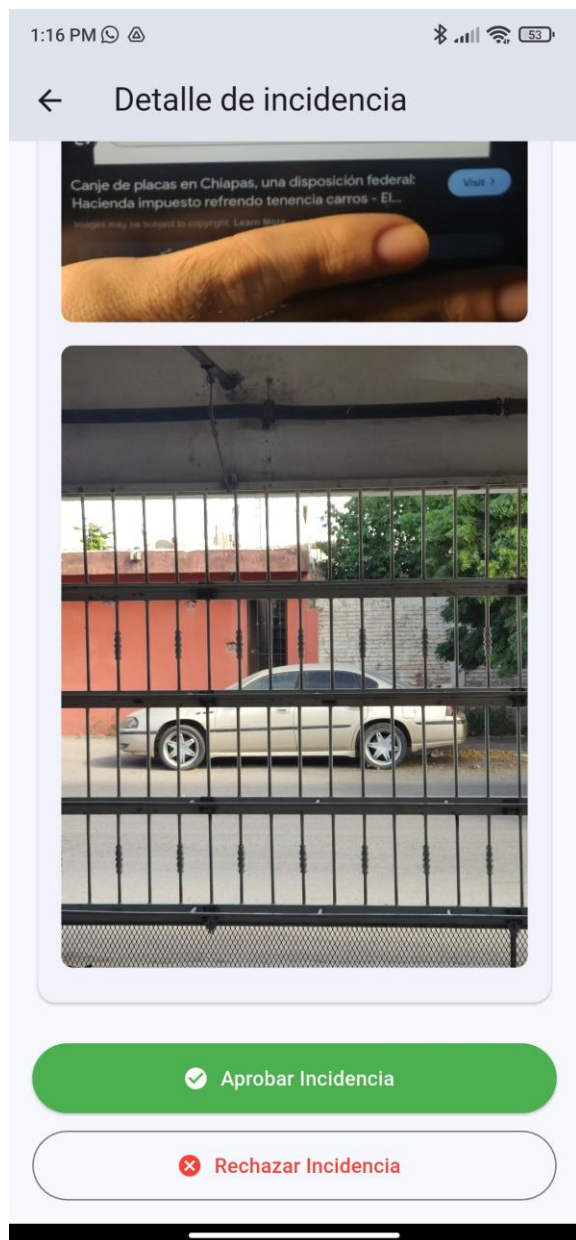
Vehículo

Placa: DSZ-999-Z

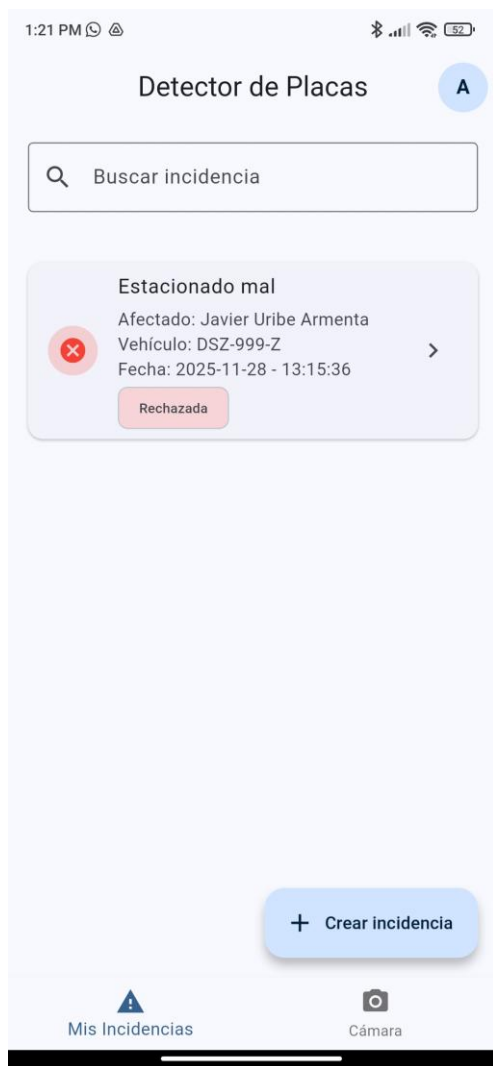
Marca: Lamborghini

Modelo: Aventador

Al presionar la tarjeta de la incidencia se pueden ver los detalles: El Estatus, Fecha y Hora, Descripción, la ubicación mostrada en Latitud y Longitud, los datos de la persona reportada, el reportante, el vehiculo y las imagenes



El administrador cuando vaya a la incidencia generada, hasta abajo va a tener dos opciones: Aprobar y rechazar, en caso de rechazar, el estatus de la incidencia cambiará a Rechazado y se enviará un correo al reportarte, informándole que su incidencia fue rechazada.



Incidencia Rechazada - Reporte #2 Inbox x



javieruribe00@outlook.com via [sendgrid.net](#)

to me ▾

Hola,

Tu reporte de incidencia ha sido revisado y RECHAZADO por el administrador.

Detalles de la incidencia:

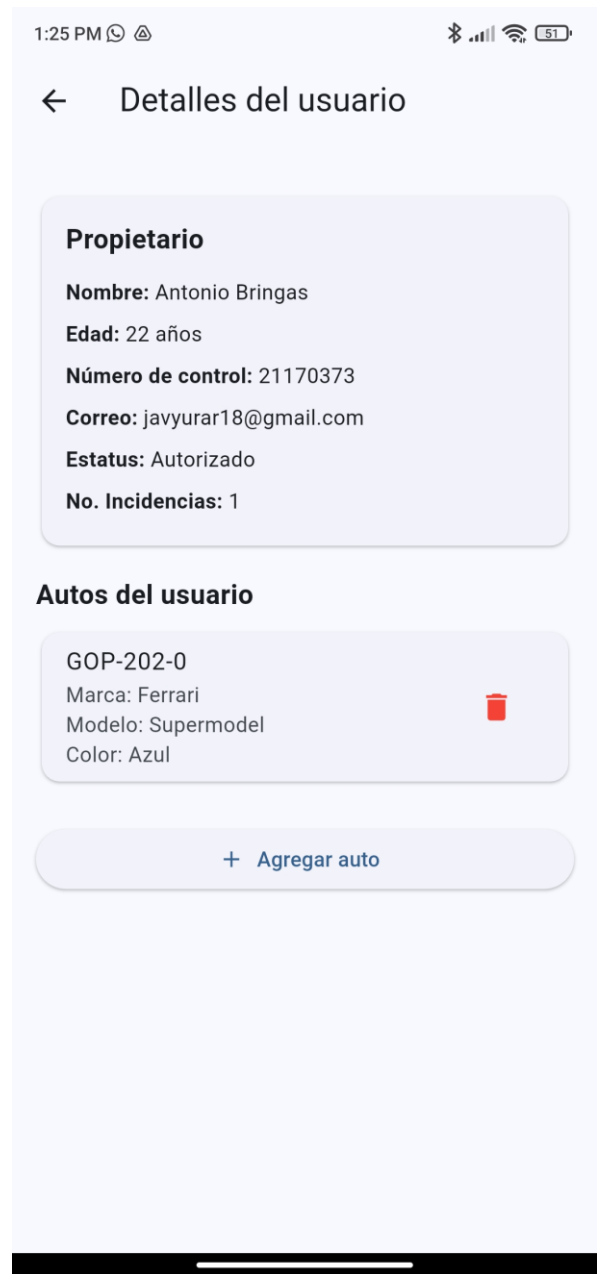
- ID: #2
- Fecha: 2025-11-28
- Hora: 13:19:27
- Descripción: jfjdjdj

La incidencia no procede según los criterios de evaluación.

Saludos,
Sistema de Control Vehicular VAFE

...

En caso de ser Aprobada, el estado de la incidencia cambiará a Aprobada, se sumará un uno al contador de incidencias de la persona reportada, y se enviarán correos tanto al reportante como al reportado, diciendo que su incidencia fue aprobada, y que se le levantó una incidencia respectivamente.



2:16 PM



Detalle de incidencia



Estatus

Aprobada

Fecha y Hora: 2025-11-28 - 13:47:41

Descripción: Mal

Ubicación: Lat: 24.7678735, Lon: -107.4610254

Persona Afectada

Nombre: Antonio Bringas

No. Control: 2117672

Correo: javyurar18@gmail.com

Estatus: Bloqueado

Incidencias totales: 3

Reportante

Nombre: Javier Uribe Armenta

No. Control: 21170497

Correo: javier.ua01@gmail.com

Vehículo

Placa: GOP-202-0

Marca: Ferrari

Notificación de Incidencia #1 - Advertencia Inbox x



javieruribe00@outlook.com via sendgrid.net
to me ▾

Estimado/a Antonio Bringas,

Se ha registrado una incidencia en tu contra:

Detalles:

- Incidencia número: 1 de 3
- ID: #3
- Fecha: 2025-11-28
- Hora: 13:25:28
- Descripción: Otra vez se estacionó en el espacio de discapacitados
- Vehículo: Ferrari - Supermodel
- Placa GOP-202-0

⚠ **ADVERTENCIA:** Actualmente tienes 1 incidencia(s) registrada(s).
Al llegar a 3 incidencias, tu acceso será bloqueado automáticamente.

Te pedimos tomar las medidas necesarias para evitar futuras infracciones.

Saludos,
Sistema de Control Vehicular VAFE

Incidencia Aprobada - Reporte #3 Inbox x



javieruribe00@outlook.com via sendgrid.net
to me ▾

Hola,

Tu reporte de incidencia ha sido aprobado por el administrador.

Detalles de la incidencia:

- ID: #3
- Fecha: 2025-11-28
- Hora: 13:25:28
- Descripción: Otra vez se estacionó en el espacio de discapacitados
- Vehículo reportado: Ferrari - Supermodel
- Placa GOP-202-0

Gracias por contribuir a mantener el orden en las instalaciones.

Saludos,
Sistema de Control Vehicular VAFE

...

[Message clipped] [View entire message](#)

Y así sucesivamente hasta que se cumplan las 3 incidencias

Notificación de Incidencia #2 - Advertencia Inbox x



javieruribe00@outlook.com via sendgrid.net
to me ▾

Estimado/a Antonio Bringas,

Se ha registrado una incidencia en tu contra:

Detalles:

- Incidencia número: 2 de 3
- ID: #4
- Fecha: 2025-11-28
- Hora: 13:29:23
- Descripción: Tapando la salida a los carros
- Vehículo: Ferrari - Supermodel
- Placa GOP-202-0

⚠ **ADVERTENCIA:** Actualmente tienes 2 incidencia(s) registrada(s).
Al llegar a 3 incidencias, tu acceso será bloqueado automáticamente.

Te pedimos tomar las medidas necesarias para evitar futuras infracciones.

Saludos,
Sistema de Control Vehicular VAFE

Incidencia Aprobada - Reporte #4 Inbox x



javieruribe00@outlook.com via sendgrid.net
to me ▾

It looks like this message is in Spanish ×
[Translate to English](#)

Hola,

Tu reporte de incidencia ha sido aprobado por el administrador.

Detalles de la incidencia:

- ID: #4
- Fecha: 2025-11-28
- Hora: 13:29:23
- Descripción: Tapando la salida a los carros
- Vehículo reportado: Ferrari - Supermodel
- Placa GOP-202-0

Gracias por contribuir a mantener el orden en las instalaciones.

Saludos,
Sistema de Control Vehicular VAFE

...

[Message clipped] [View entire message](#)

ACCESO BLOQUEADO - Tercera Incidencia Registrada Inbox x



javieruribe00@outlook.com via sendgrid.net
to me ▾

Estimado/a Antonio Bringas,

Se ha registrado tu tercera incidencia y tu acceso ha sido BLOQUEADO:

Detalles de la última incidencia:

- ID: #9
- Fecha: 2025-11-28
- Descripción: Mal
- Vehículo: Ferrari Supermodel - Placa GOP-202-0

 ESTADO: BLOQUEADO

Has acumulado 3 incidencias, por lo que tu acceso a las instalaciones ha sido suspendido.

Para solicitar la reactivación de tu acceso, por favor contacta al administrador.

Saludos,
Sistema de Control Vehicular VAFE

Incidencia Aprobada - Reporte #9 Inbox x



javieruribe00@outlook.com via sendgrid.net
to me ▾



It looks like this message is in Spanish



[Translate to English](#)

Hola,

Tu reporte de incidencia ha sido aprobado por el administrador.

Detalles de la incidencia:

- ID: #9
- Fecha: 2025-11-28
- Hora: 13:47:41
- Descripción: Mal
- Vehículo reportado: Ferrari - Supermodel
- Placa GOP-202-0

Gracias por contribuir a mantener el orden en las instalaciones.

Saludos,
Sistema de Control Vehicular VAFE

...

[Message clipped] [View entire message](#)

Una vez cumplidas las 3 incidencias, el estatus del usuario cambiará de Autorizado a Bloqueado.

← Detalles del usuario

Propietario

Nombre: Antonio Bringas

Edad: 22 años

Número de control: 21170373

Correo: javyurar18@gmail.com

Estatus: Bloqueado

No. Incidencias: 3

Autos del usuario

GOP-202-0

Marca: Ferrari

Modelo: Supermodel

Color: Azul



+ Agregar auto

Manual de Instalación

Nota: Se intentó usar un servidor de Render para utilizar el backend desde ahí y no depender de un servidor local, pero el tamaño del servidor no es suficiente para las peticiones que hace el servidor (puesto que se usa una dependencia de Python que llama a la API de PaddleOCR) por lo que, si se puede tener un servidor con más memoria, solo sería cambiar una línea de código para que la app móvil apunte a esa dirección para hacer las peticiones, es decir, solo se tendría que descargar el APK y abrir la aplicación, pero como ese no es el caso, se tiene que correr de manera local.)

Primero se debe clonar el repositorio:

```
git clone https://github.com/EIJavi115/VAFE_Plate_Detector.git
```

Una vez clonado el repositorio se debe acceder a la carpeta del proyecto, estando allí se deben hacer dos cosas:

Ejecutar el backend:

Como todo está en contenedores de Docker, no se necesita instalar ninguna dependencia. Así que, todo lo que se tiene que hacer es instalar Docker.

Una vez instalado Docker, se tiene que ir a la carpeta “backend” del proyecto, abrir una terminal y ejecutar los siguientes comandos:

Para construir los contenedores usar:

```
docker compose up --build
```

(Si ya están contruidos y solo se quieren activar, usar docker compose up)

Una vez levantados los contenedores, en otra terminal verificar cual es su dirección IP, pues es esta a donde se harán las peticiones.

En Windows:

```
show ip address
```

En Linux:

```
ip addr show
```

Nota: No copiar el prefijo de la IP, solo la IP.

Modificar y ejecutar el Frontend:

Nota: Se debe tener instalado flutter en su sistema, para eso siga las instrucciones de la documentación oficial de Flutter: <https://docs.flutter.dev/install/manual>

Una vez obtenida la IP, se debe dirigir a:

```
Detector de Placas/mobile/plate_detector/lib/services/api_client.dart
```

y cambiar la linea de código 15:

```
final String _baseUrl = 'http://{{TU IP}}:8000';
```

Reemplazar {{TU IP}} por tu ip.

Una vez hecho esto se debe generar el APK, para esto se puede hacer de dos maneras:

Generándola mediante Flutter:

Abrimos una consola y nos situamos en Detector de Placas/mobile/plate_detector, si ya estábamos situados en api_client no necesitamos volvernos a mover.

Primero nos aseguramos de que tenemos todas las dependencias con:
`flutter pub get`

Luego, ejecutamos el siguiente comando para generar el apk:
`flutter build apk --release`

El APK generado estará en esta dirección:
Detector de Placas/mobile/plate_detector /build/app/outputs/flutter-apk/app-release.apk

Una vez hecho esto, pasar el apk a su dispositivo e instalar.

Generándola al debuggear el código:
Podemos utilizar nuestro dispositivo móvil como dispositivo para debuggear/probar nuestro código, entonces podemos usar nuestro dispositivo conectándolo con un cable activando la transferencia de archivos. Una vez teniendo todo listo, ejecutar la línea:
`flutter run`

Dentro de la carpeta del proyecto de flutter, y el apk se construirá en el dispositivo móvil.

Documentación Técnica

Backend:

Dependencias utilizadas:

- fastapi
- uvicorn[standard]
- SQLAlchemy
- psycopg2-binary
- python-multipart
- pydantic
- pydantic[email]
- paddleocr
- opencv-python
- sendgrid
- python-dotenv

El backend se construyó utilizando la librería de FastAPI, para hacer las llamadas entre el servidor y la aplicación móvil. Utilizando dos contenedores de docker para almacenar la base de datos de PostgreSQL y la API, se tienen 3 archivos, el dockercompose, que es de donde se crean los contenedores y el DockerFile, que corre los scripts para instalar las dependencias de python y luego correr el backend.

Para hacer los modelos utilizados para hacer las tablas en la BD, se utilizó sqlalchemy que a través de sus librerías nos permite hacer una tabla con todos sus atributos de forma sencilla. Para realizar los schemas (modelos que se utilizan para mandar JSON) utilizamos pydantic, con BaseModel para crear los modelos y EmailStr para validar el string del correo en el caso de los usuarios, se hicieron dos schemas, uno para leer los datos, es decir, agregarlos, y otro para leerlos, osea para obtener la información de ellos.

Para el servicio de correos, nos conectamos a la API de SendGrid, que nos permite tener un sistema de administración de correos, y la posibilidad hacer plantillas y también enviarlos mediante un dominio de correo personalizado.

Nota: Al estar enviando correos de esta manera, puede que algunos correos los bloquee el cliente de correo electrónico como Gmail o Outlook, los bloquee. Es muy mala práctica el poner tus credenciales en el código fuente, es por eso que utilizamos un .env para guardar la API_KEY y llamarla cargando el env y leyendola variable de entorno.

Frontend:

Dependencias utilizadas:

- cupertino_icons: ^1.0.8
- http: ^1.6.0
- paddle_ocr: ^0.0.5
- tf_lite_flutter: ^0.12.1
- path_provider: ^2.1.5
- image: ^3.3.0
- camera: ^0.11.3
- shared_preferences: ^2.5.3
- image_picker: ^1.2.1
- intl: ^0.20.2
- flutter_launcher_icons ^0.11.0

Por el lado de la aplicación web las dependencias utilizadas, fueron más que nada para el diseño y funcionalidad el frontend, el poner imágenes, utilizar la cámara, y crear widgets dentro de la app.

- cupertino_icons: Nos da acceso a una lista de iconos de iOS.
- http: Nos sirve para hacer peticiones HTTP (GET, POST, PATCH, etc.)
- paddle_ocr: Nos sirve para hacer el reconocimiento de texto en las imágenes que estamos tomando con la cámara
- tf_lite_flutter: Nos permite utilizar tensorflow lite en Flutter y permite la ejecución de la red neuronal en el dispositivo móvil.
- path_provider: Nos sirve para saber en que carpeta guardar o leer los archivos en el dispositivo.
- image: Nos permite manipular imágenes (redimensionar, recortar, convertir formatos, etc.)
- camera: Nos da acceso a la cámara del dispositivo
- shared_preferences: Sirve como un “cache” en el dispositivo, para guardar por ejemplo el token de inicio de sesión por poner un ejemplo.

- `image_picker`: Permite seleccionar imágenes o videos desde la galería o la cámara
- `intl`: Es una librería de internacionalización y formatos. La usamos para el tema de poner la fecha y hora en la incidencia
- `flutter_launcher_icons`: Lo usamos para ponerle un logo al apk generado

Lo importante por mencionar es el procesamiento del modelo de detección de placas

El flujo de detección de placas inicia directamente en la aplicación móvil, en la sección de Cámara o desde la pantalla de Creación de Incidencia, cuando el usuario pulsa el botón para escanear una placa.

1. Captura de la imagen con la cámara

Al ingresar a la sección de Cámara, la aplicación solicita los permisos necesarios para utilizar la cámara del dispositivo. Una vez otorgados, se muestra una vista previa con un rectángulo guía donde el usuario debe posicionar la placa del vehículo lo mejor posible.

Al presionar el botón de captura, se toma una fotografía y esta imagen se procesa dentro del propio dispositivo.

2. Ejecución del modelo de detección de placas en el dispositivo

La imagen capturada se envía al modelo de detección de placas, que fue entrenado con Ultralytics YOLOv8 y exportado a formato TFLite para poder ejecutarse de forma eficiente en dispositivos móviles.

- El modelo recibe la imagen completa.
- Realiza el procesamiento convolucional (típico de una CNN) para detectar objetos del tipo “*Patente*” dentro de la imagen.
- Como salida, el modelo genera una o varias cajas delimitadoras (bounding boxes) con la ubicación de las placas detectadas y un puntaje de confianza.

3. Recorte de la región de interés (ROI) de la placa

Una vez obtenida la caja con mayor confianza, la aplicación recorta la porción de la imagen donde se encuentra la placa.

Este recorte se realiza usando las coordenadas devueltas por el modelo (x, y, ancho y alto de la caja) y produce una nueva imagen más pequeña que contiene únicamente la placa. Esta imagen recortada es la que se utilizará para la etapa de reconocimiento de caracteres.

4. Preparación de la imagen para el backend

La imagen recortada (la placa) se convierte a un formato adecuado (por ejemplo JPG/PNG en memoria) y se empaqueta en una petición HTTP (tipo multipart/form-data). La aplicación envía esta imagen al backend a través de un endpoint específico

para OCR de placas, junto con la información necesaria (por ejemplo, cabeceras, tipo de contenido, etc.).

Proceso de detección de caracteres con PaddleOCR (backend)

La segunda parte del flujo ocurre en el backend, dentro del contenedor de la API construido con FastAPI y Docker. Aquí es donde entra en juego PaddleOCR para convertir la imagen de la placa en texto.

1. Recepción de la imagen recortada en el backend

El backend expone un endpoint dedicado para la lectura de placas mediante OCR.

- Al recibir la petición, FastAPI obtiene el archivo enviado por la app móvil.
- La imagen se lee en memoria y se decodifica utilizando OpenCV para obtener una matriz de píxeles (imagen en formato numérico).

2. Preprocesamiento ligero y uso de archivo temporal

Para facilitar el procesamiento por parte de PaddleOCR, la imagen se puede guardar momentáneamente en un archivo temporal en el servidor. Esto permite que la librería de OCR procese la ruta del archivo directamente sin tener que modificar la forma en la que trabaja internamente.

3. Ejecución de PaddleOCR sobre la imagen de placa

Con la imagen lista, se invoca a PaddleOCR indicando el idioma y las configuraciones necesarias.

PaddleOCR realiza las siguientes tareas:

- Analiza la imagen recortada de la placa.
- Detecta las regiones que contienen texto (en este caso, los caracteres de la matrícula).
- Reconoce, para cada región, la secuencia de caracteres (letras y números) y asigna un score de confianza. El resultado es una lista de textos reconocidos junto con sus probabilidades.

4. Filtrado y normalización del texto de la placa

Una vez que el backend tiene todas las cadenas detectadas por PaddleOCR, se realiza una etapa de postprocesamiento para quedarse con la mejor candidata como placa:

- Se normaliza el texto: se pasa a mayúsculas, se eliminan espacios y caracteres innecesarios.
- Se descartan cadenas demasiado cortas o largas para ser una placa válida.
- Se verifican criterios simples, como que la cadena tenga al menos una letra y un número.

- Se ordenan los candidatos por su score de confianza y se selecciona el más probable.

Como resultado, el backend obtiene:

- El texto crudo que devolvió el OCR.
- La placa normalizada (por ejemplo, sin espacios).
- El score asociado.

5. Búsqueda de la placa en la base de datos

Con la placa normalizada, el backend realiza una consulta a la base de datos PostgreSQL:

- Se busca un registro de Auto cuya placa coincida con la cadena reconocida.
- Si se encuentra un auto, se recupera también la información de la Persona a la que pertenece (nombre, número de control, correo, estatus, número de incidencias, etc.).
- Si no se encuentra ninguna coincidencia, el backend indica que no hay match en la base de datos.

6. Construcción de la respuesta hacia la app

El backend arma una respuesta en formato JSON con dos partes principales:

- Un bloque con la información del OCR: texto detectado, placa normalizada y score.
- Un bloque opcional con los datos del auto y de la persona, en caso de que exista un registro en la base de datos asociado a esa placa.

Esta respuesta se envía de vuelta a la aplicación móvil.

Integración de ambos procesos en la aplicación

Cuando la aplicación recibe la respuesta del backend:

1. Si la placa fue reconocida y existe en la base de datos, la app:
 - Autocompleta los campos de propietario (nombre, número de control, correo) y del vehículo (marca, modelo, color) en las pantallas correspondientes (por ejemplo, detalle de usuario o creación de incidencia).
 - Muestra un mensaje indicando que la placa fue encontrada exitosamente.
2. Si la placa fue reconocida pero no está registrada en el sistema, la app:
 - Muestra un mensaje al usuario indicando que la placa no se encuentra en la base de datos.

- Permite que el administrador registre el auto y lo asocie a una persona, o que el usuario proceda de forma manual.
3. Este flujo de detección de placa + reconocimiento de caracteres reduce el trabajo manual del usuario, ya que basta con apuntar la cámara a la placa para obtener automáticamente los datos relacionados al vehículo y su dueño, integrando la parte de detección de objetos (YOLOv8-TFLite) en el dispositivo con la etapa de reconocimiento de texto (PaddleOCR) en el backend.

Información acerca del modelo

<https://universe.roboflow.com/cerda/tesis-vosk3>

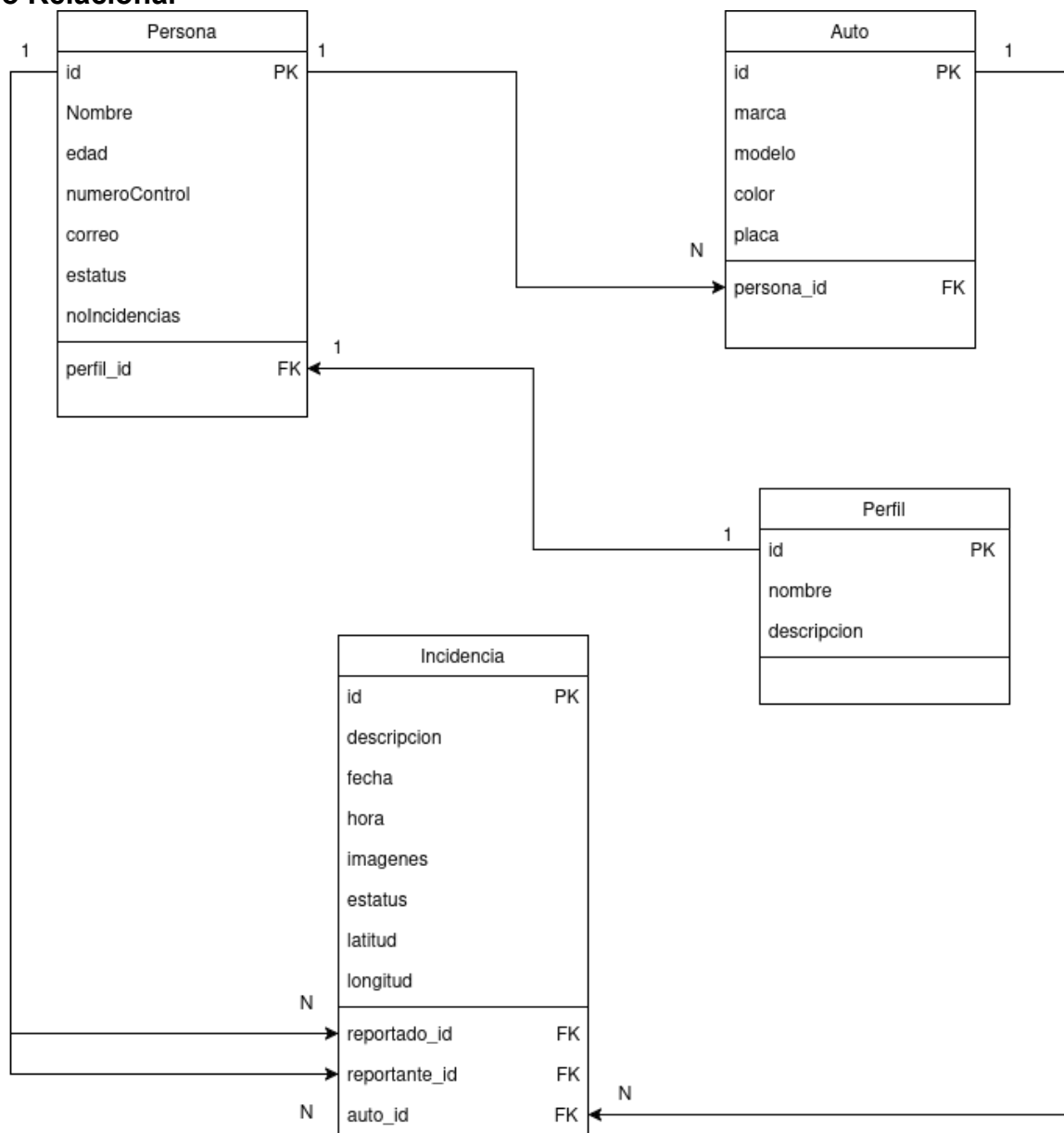
El modelo es Ultralytics YOLOv8 utiliza una arquitectura CNN (Convolutional Neuronal Network) para detección de objetos. Y como formato es un tflite para utilizarlo en dispositivos móviles, al ser más ligero. Consta de 1738 imágenes, y ya tiene sus aumentaciones.

Para la validación del modelo de detección de placas se utilizó el módulo de evaluación de YOLOv8 sobre el conjunto de validación definido en el dataset. YOLO calcula de forma automática métricas estándar como precisión, recall y *mean Average Precision* (mAP), donde el mAP se interpreta como el área bajo la curva precisión–recall para la clase “Patente” que era la que tenía el modelo.

En nuestro caso, el modelo obtuvo una precisión aproximada de 0.99, un recall de 0.997 y un mAP@0.5 cercano a 0.995, lo que indica que el detector presenta un desempeño alto y estable en la identificación de placas dentro de las imágenes.

```
fitness: np.float64(0.8450947227989565)
keys: ['metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)', 'metrics/mAP50-95(B)']
maps: array([ 0.84509])
names: {0: 'Patente'}
nt_per_class: array([352])
nt_per_image: array([348])
results_dict: {'metrics/precision(B)': 0.9903132993048965, 'metrics/recall(B)': 0.9971590909090909, 'metrics/mAP50(B)': 0.994712598809821, 'metrics/mAP50-95(B)': 0.8450947227989565,
'fitness': 0.8450947227989565}
save_dir: PosixPath('/content/runs/detect/val')
speed: {'preprocess': 2.9700588477017043, 'inference': 8.718037557466085, 'loss': 0.000867620677116525, 'postprocess': 1.2186857471303598}
stats: {'tp': [], 'conf': [], 'pred_cls': [], 'target_cls': [], 'target_img': []}
task: 'detect'
```

Modelo Relacional



Una persona puede tener muchos autos: Persona 1 – N Autos

Una persona solo puede tener un perfil: Persona 1 – 1 Perfil

Un reportante puede hacer N incidencias: Persona (Reportante) 1 – N Incidencia

Un reportado/afectado puede tener N incidencias: Persona (Reportado/Afectado) 1 – N Incidencias

Un auto puede estar en N incidencias: Auto 1 – N Incidencia

Conclusiones.

El sistema que se ha desarrollado permite automatizar la gestión vehicular dentro del Instituto Tecnológico de Culiacán mediante la implementación eficiente entre visión artificial, OCR, bases de datos y una aplicación móvil desarrollada de tal manera que sea intuitiva para el usuario. El proyecto presenta una solución viable que mejora la seguridad, reduce procesos manuales y permite el registro de incidencias con información precisa y verificable.

- Link al repositorio escolar para revisión de proyecto:
<https://github.com/EIJavi115/Topicos-de-IA>
- Link al repositorio original: https://github.com/EIJavi115/VAFE_Plate_Detector
- Link al video demostración:
https://drive.google.com/file/d/1dlyl_D0bHSI8CDJHM0harhVJeFy7K48o/view?usp=sharing